

Analyzing the Impact of Activation Quantization on Sparse Autoencoder Representations

Nidheesh Kannadasan

June 7, 2026

1 Introduction

Large Language Models (LLMs) rely on internal activation representations to process and store information. To reduce memory usage and computational cost, these activations are often quantized to lower numerical precision. Although quantization can significantly improve efficiency, it may also distort internal representations and reduce model performance. Most quantization studies evaluate this trade-off using downstream metrics such as perplexity. While useful, these metrics do not explain how quantization affects the model internally. In this project, we investigate quantization at the representation level using Sparse Autoencoders (SAEs). By training SAEs on hidden activations from DistilGPT2, we obtain a sparse feature representation that can be analyzed directly. The goal of the project is to identify which features are most affected by quantization, measure how representation geometry changes under different bitwidths, and evaluate whether the most distorted features are also the most important for model behavior. Based on these observations, we implement and evaluate a subspace-preserving quantization strategy designed to better retain important information.

2 Computational Setup & Framework

2.1 Hardware Constraints

All experiments, training loops, and evaluation pipelines were executed locally on an NVIDIA RTX 4060 GPU with a strict hardware limit of 8GB VRAM running on an Ubuntu Linux environment.

2.2 Dataset Strategy

The assignment required several evaluation metrics with significantly different computational costs. As a result, separate datasets were allocated for training, calibration, perplexity evaluation, geometric analysis, and causal importance measurements. This allowed each experiment to be performed efficiently while remaining within the available hardware constraints:

1. **409.6M Top-K SAE Train Set:** Used for training the Top-K SAE. Note that in the assignment it was mentioned to use roughly 1% of the corpus (80-100M tokens), a batch size between 4k-8k and 100k steps. This created a three-way circular dependency which could not be met. In order to resolve this and considering 100k steps as a proxy

for training duration, a batch size of 4096 was chosen due to computational constraints, leading to a train set of 409.6M tokens.

2. **204.8k tokens calibration set:** This dataset, a subset of the training set consisting of exactly 204,800 tokens, was used to compute min/max calibration statistics for Tasks 2 and 4.
3. **2M Tokens Held-out Stream:** Used exclusively for computing downstream Perplexity where a large sample size is required for statistical convergence.
4. **10112 Tokens held out set:** Used to visualize geometry with UMAP.
5. **4096 Tokens Held-out Batch:** Used for computing geometric and spectral metrics, including Singular Value Decomposition (SVD), Subspace Distortion Scores (SDS), Mean Squared Error (MSE), and Centered Kernel Alignment (CKA). Because the computational complexity of SVD scales as $\mathcal{O}(Nm^2)$, executing it on the full 2M token stream would instantly trigger an out-of-memory error.
6. **50k Tokens Held-out Subset:** Used for computing causal importance metrics (Fisher-style gradients). This subset is randomly sampled from a highly homogeneous partition of the OpenWebText corpus. The expected value of local gradients converges efficiently on this allocation, providing a reliable proxy for causal impact without requiring large-scale backward passes.

3 Task 1: Pipeline Setup & Sparse Autoencoder Training

3.1 Data Extraction & Preprocessing

Using the `TransformerLens` library, we initialized a frozen, pretrained `distilgpt2` model. We streamed tokens from a subset of the OpenWebText corpus. Hidden states were intercepted and extracted from Layer 2 (0-indexed, corresponding to the third layer of the 6-layer architecture). We restricted extraction to contiguous sequences of exactly 128 tokens, omitting all padded sequences and isolated-token samples. Prior to autoencoder training, the gathered hidden states (structured as `[batch, seq_len, 768]`) were normalized and completely shuffled along the batch and sequence dimensions to disrupt temporal correlations across adjacent tokens.

3.2 SAE Architecture & Optimization

We trained Top- K Sparse Autoencoders via the `SAELens` library to map the dense layer activations into an expanded, sparse feature space. The autoencoder restricts activation density by enforcing a hard 10% sparsity constraint ($k = 0.1m$). We evaluated two dictionary bottleneck capacities: $m \in \{512, 1024\}$, initializing training with $m = 512$. The forward pass, dictionary encoding, and reconstruction are governed by the following mathematical formulations:

$$z = E(x), \quad \hat{x} = D(z), \quad \mathcal{L}_{\text{SAE}} = \|x - \hat{x}\|_2^2 \quad (1)$$

$$\text{subject to } |\text{supp}(z)| \leq k, \quad \text{where } k = 0.1m \quad (2)$$

Optimization was executed using the Adam optimizer with a fixed learning rate $\eta = 10^{-4}$ and an operational batch size of 4096. The optimization trajectory ran for exactly 100,000 steps.

3.3 Weights & Biases Monitoring Integration

All core training dynamics were streamed directly to Weights & Biases (W&B). Key tracked metrics included the structural Reconstruction Loss (MSE), Sparsity Loss profiles, the explicit stability of the L_0 norm, and dead neuron counts.

The training logs, loss curves and metrics can be accessed directly via the following public workspace links:

- Weights & Biases Run Workspace — $m = 512$ Bottleneck Capacity
- Weights & Biases Run Workspace — $m = 1024$ Bottleneck Capacity

4 Task 2: Quantization Analysis & Activation Distortion

4.1 Quantization Framework and Intervention Strategy

We used uniform, linear quantization to test how well the learned dictionary spaces handle precision drops. We tested per-tensor and per-feature configurations across a sweep of bitwidths ($b \in \{8, 6, 4, 2\}$). We chose an unsigned configuration ranging from $[0, 2^b - 1]$ because the Top- K SAE architecture uses a ReLU activation function, meaning all valid feature activations are already non-negative ($z \geq 0$). Designing a signed grid would waste bits on negative values that do not exist.

Scaling steps (Δ) are calculated dynamically from min/max statistics collected across a 204.8k token calibration stream, using a tiny lower bound of 10^{-8} to avoid dividing by zero. The operations scale vectors according to:

$$\Delta_{\text{per_tensor}} = \max\left(\frac{z_{\text{global_max}}}{2^b - 1}, 10^{-8}\right), \quad \Delta_{\text{per_feature},j} = \max\left(\frac{z_{\text{feat_max},j}}{2^b - 1}, 10^{-8}\right) \quad (3)$$

$$\hat{z} = \text{clip}\left(\left\lfloor \frac{z}{\Delta} \right\rfloor, 0, 2^b - 1\right), \quad \tilde{z} = \Delta \hat{z} \quad (4)$$

Interventions are executed directly on the Layer 2 residual stream hook location. Full-precision activations are intercepted, mapped to sparse representations ($z = E(x)$), quantized to form $\tilde{z}$, and reconstructed via the decoder ($\hat{x} = D(\tilde{z})$) before the model resumes its forward pass. Geometric distortions are evaluated via Mean Squared Error (MSE), Subspace Distortion Scores (SDS), and Centered Kernel Alignment (CKA) using double precision to prevent numerical overflow:

$$\text{CKA}(\mathbf{X}, \mathbf{Y}) = \frac{\|\tilde{\mathbf{X}}^\top \tilde{\mathbf{Y}}\|_F^2}{\|\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}\|_F \|\tilde{\mathbf{Y}}^\top \tilde{\mathbf{Y}}\|_F} \quad \text{where} \quad \tilde{\mathbf{X}} = \frac{\mathbf{X}_c}{\|\mathbf{X}_c\|_F + 10^{-8}} \quad (5)$$

4.2 Results and Interpretation

Comparing the geometric metrics against downstream performance highlights clear differences in how the two quantization approaches break down.

Subspace Distortion Dynamics (SDS_k)

Per-tensor quantization causes a sharp jump in subspace distortion (SDS_k) when precision drops below 6 bits, especially in larger subspaces like $k = 128$. On the other hand, per-feature quantization keeps distortion low and stable (around 0.04) all the way down to 2 bits. This works because it calculates independent scaling factors for each individual feature rather than forcing one global scale across the whole tensor.

Representation Alignment and the 2-Bit CKA Anomaly

Model perplexity gets significantly worse under per-tensor quantization, tracking a sharp drop in CKA similarity down to 4 bits. Surprisingly, at 2 bits, the per-tensor CKA score shows an artificial "bounce" upward. This happens because the coarse global scale rounds most smaller features to zero. The few remaining active features collapse into a simple, co-linear layout. Since CKA uses normalized dot products, this collapse accidentally inflates the similarity score, making it look like a recovery when the model is actually broken. Per-feature quantization completely avoids this issue.

Geometric Reconstruction Accuracy (MSE)

The feature reconstruction error (MSE) tracks perfectly with the drop in model performance (perplexity). Rounding errors introduced right at the autoencoder bottleneck grow and compound rapidly as the forward pass runs through the rest of the model's layers.

Dictionary Capacity Tradeoffs and Sparsity Fragility

At high precision (8-bit), the larger dictionary size ($m = 1024$) successfully captures dense features. However, at low bitwidths using per-tensor quantization, it fails completely due to "Sparsity Fragility." The larger dictionary splits concepts into very sparse, high-magnitude features. These extreme outlier activations inflate the global scaling factor, which forces regular, lower-magnitude features to round down to zero and deletes important information.

High-Dimensional Manifold Visualization via UMAP

UMAP plots show that feature clusters completely disappear under 4-bit and 2-bit per-tensor quantization, visually proving the feature collapse responsible for the CKA bounce anomaly. In contrast, per-feature scaling keeps the cluster structures intact down to 2 bits, keeping errors localized to a few busy hubs while leaving the rest of the feature layout clean.

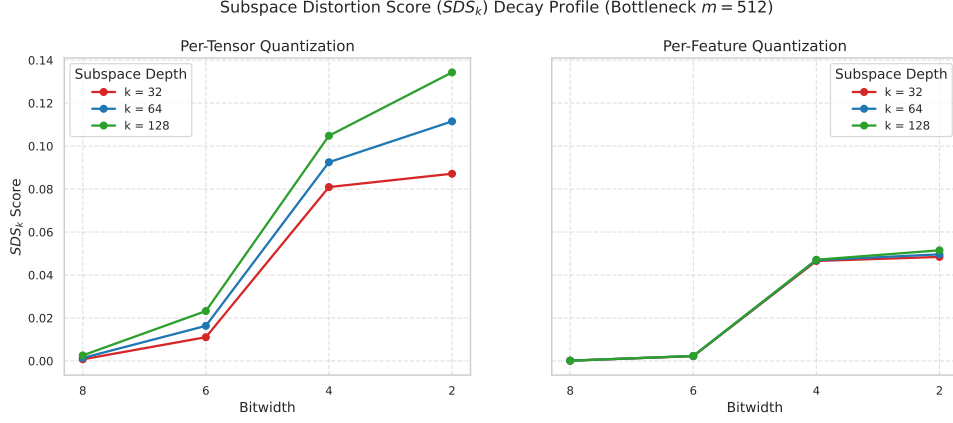


Figure 1: Subspace Distortion Score (SDS_k) decay profile for the $m = 512$ dictionary bottleneck across varying subspace depths ($k \in \{32, 64, 128\}$). The left panel illustrates per-tensor quantization, showing severe non-linear distortion accumulation at low bitwidths. The right panel displays the robust suppression of subspace leakage achieved via per-feature quantization.

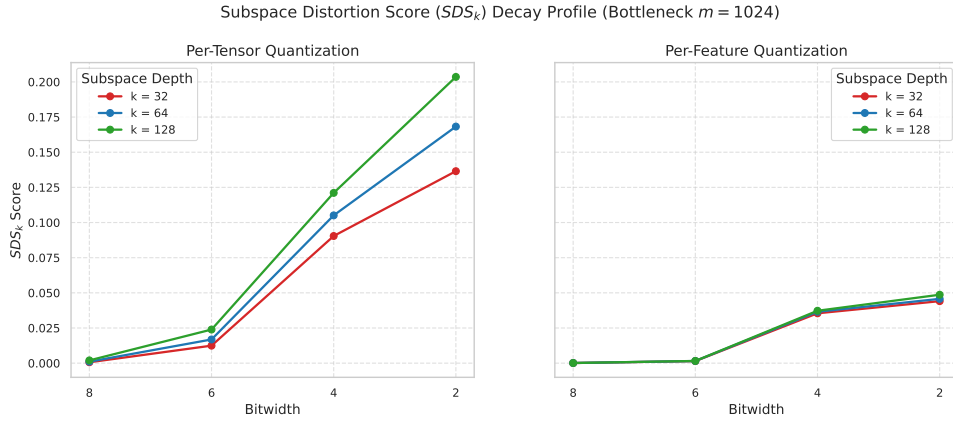


Figure 2: Subspace Distortion Score (SDS_k) decay profile for the $m = 1024$ dictionary bottleneck. Comparing this to the $m = 512$ configuration highlights that an expanded dictionary space suffers from amplified absolute subspace leakage under per-tensor schemes, peaking above 0.20 at 2-bit precision.

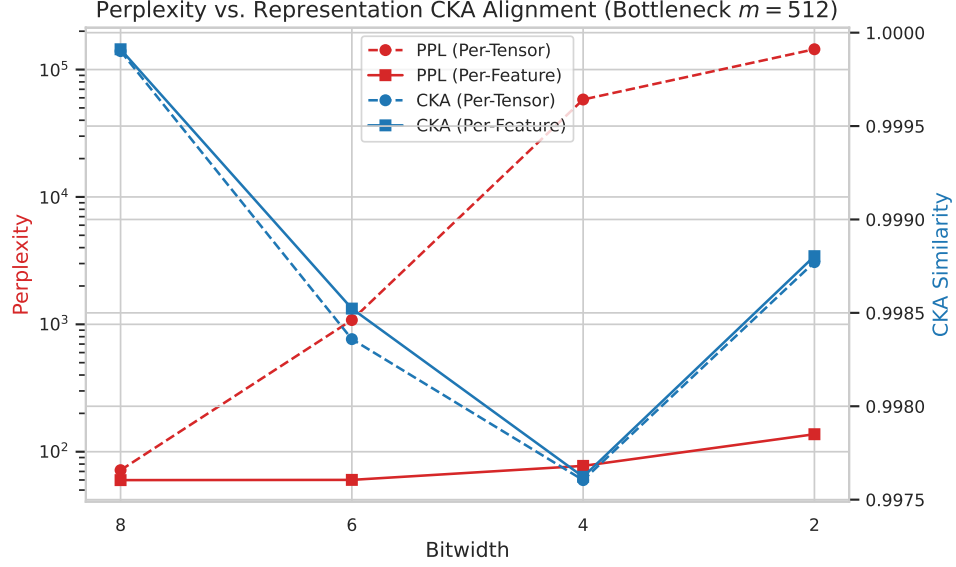


Figure 3: Downstream Perplexity vs. Representational CKA Alignment for the $m = 512$ dictionary capacity. The dual-axis plot illustrates the exponential explosion of per-tensor perplexity matching a drop in CKA down to 4-bit, followed by the characteristic 2-bit CKA "bounce" induced by low-variance dimensional collapse.

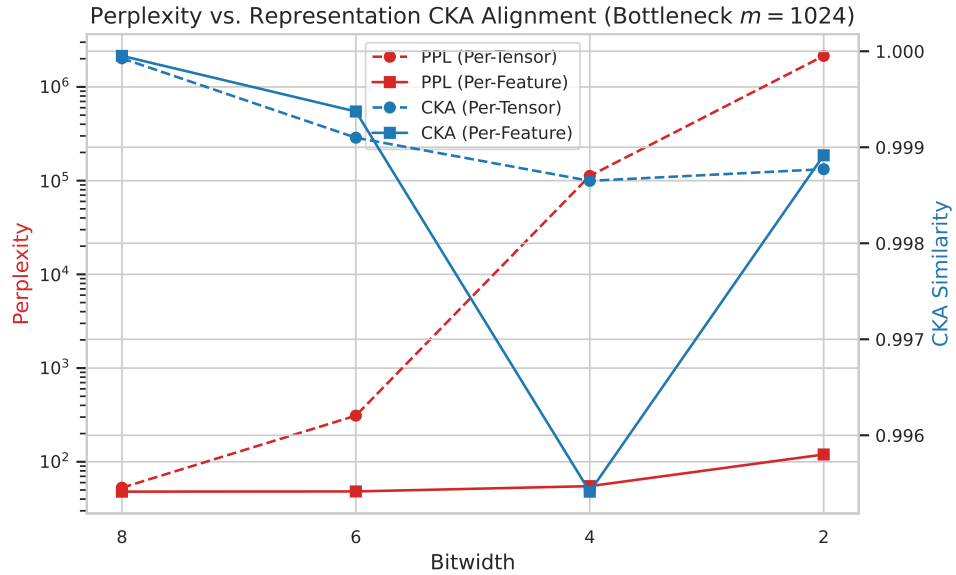


Figure 4: Downstream Perplexity vs. Representational CKA Alignment for the $m = 1024$ dictionary capacity. The expanded bottleneck experiences more severe degradation under per-tensor quantization, while per-feature quantization preserves stable representational mapping and perplexity values.

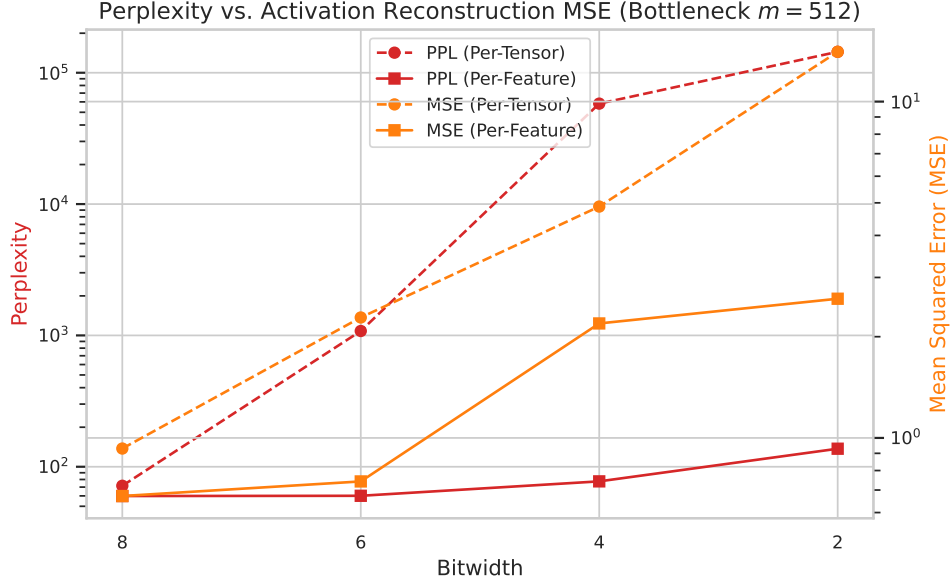


Figure 5: Downstream Perplexity vs. Activation Reconstruction Mean Squared Error (MSE) for the $m = 512$ model. The plot shows a clear monotonic tracking between local bottleneck coordinate distortion and macro-level language modeling performance degradation.

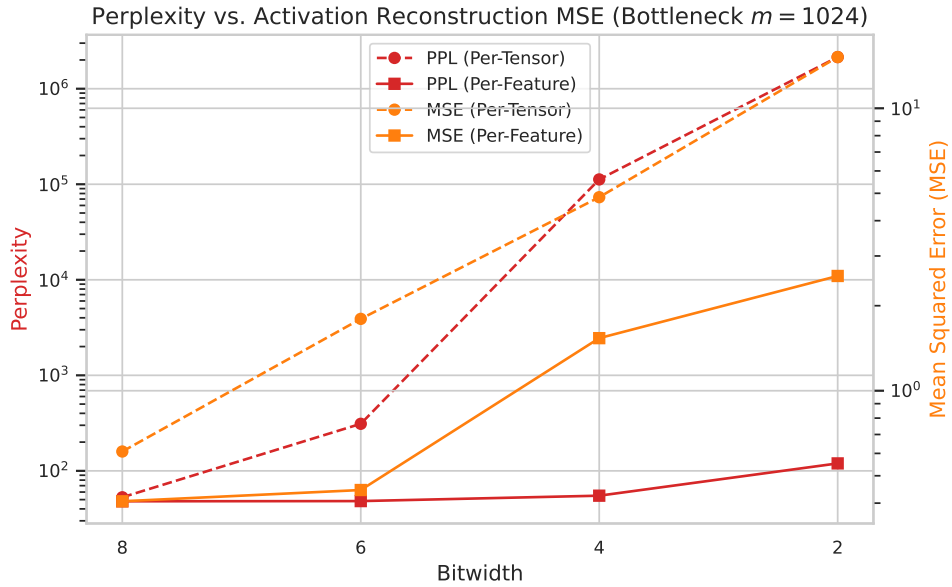


Figure 6: Downstream Perplexity vs. Activation Reconstruction Mean Squared Error (MSE) for the $m = 1024$ model. The per-tensor framework shows an accelerated accumulation of geometric reconstruction error compared to the highly stable per-feature pipeline.

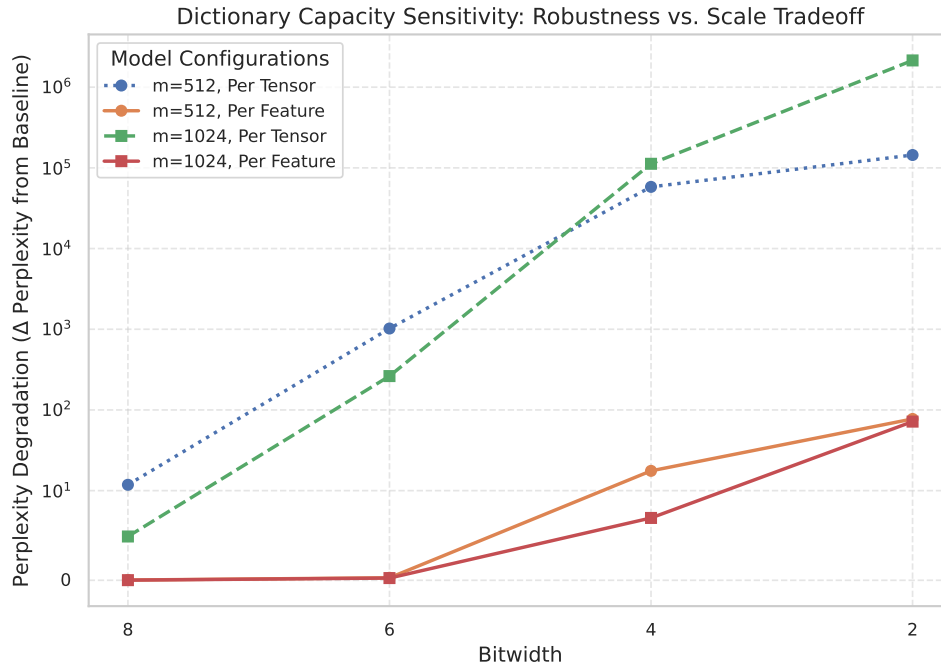


Figure 7: Dictionary Capacity Sensitivity Mapping (Δ Perplexity from Baseline) across all evaluated configurations. The Pareto curves demonstrate that while larger dictionary capacities ($m = 1024$) match or exceed smaller configurations at higher precisions, they exhibit severe "Sparsity Fragility" under low-bit per-tensor quantization.

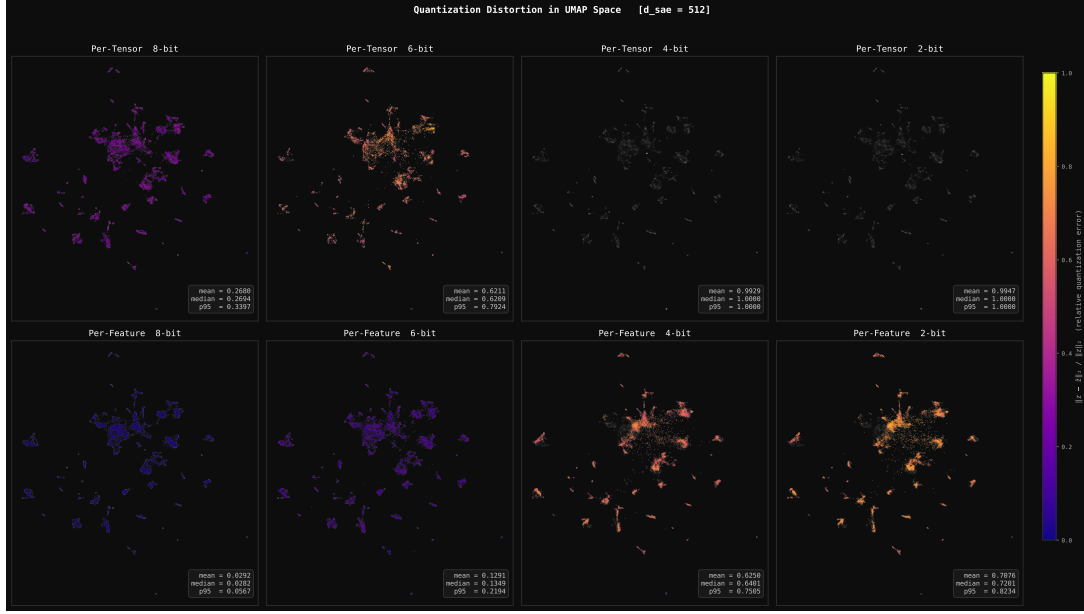


Figure 8: Quantization distortion manifold mapping in UMAP space for the $m = 512$ dictionary capacity. Points are colored by relative quantization error. The top row illustrates the immediate error saturation (median error = 1.0000) across the entire representation space under 4-bit and 2-bit per-tensor configurations, whereas the bottom row demonstrates graceful, localized error degradation under the per-feature framework.



Figure 9: Quantization distortion manifold mapping in UMAP space for the $m = 1024$ dictionary capacity. Similar to the $m = 512$ baseline, the per-feature implementation effectively prevents global error saturation, maintaining a median distortion floor of 0.6627 even at 2-bit precision.

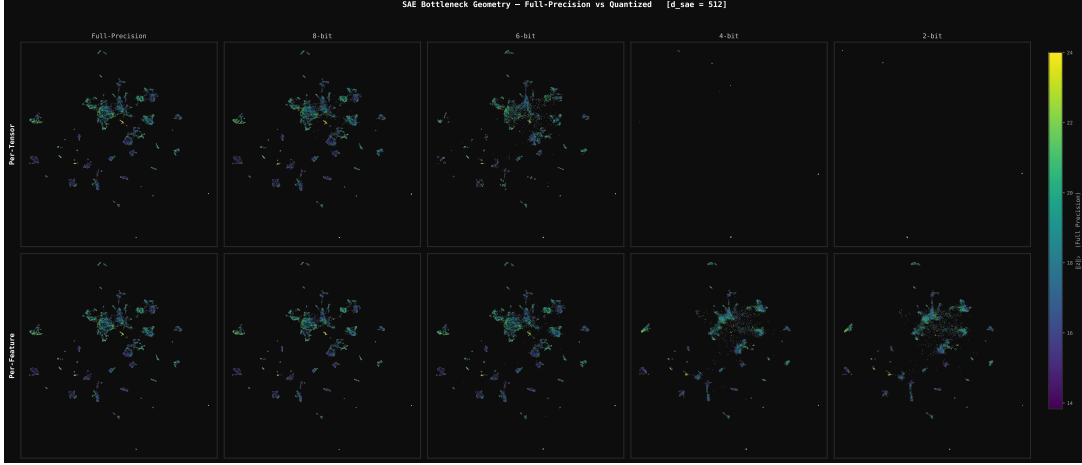


Figure 10: SAE bottleneck geometry and structural cluster comparison (Full-Precision vs. Quantized) for $m = 512$. The top row shows the catastrophic topological collapse and erasure of feature activations under low-bit per-tensor quantization, while the bottom row demonstrates the robust preservation of the global manifold skeleton via per-feature calibration.

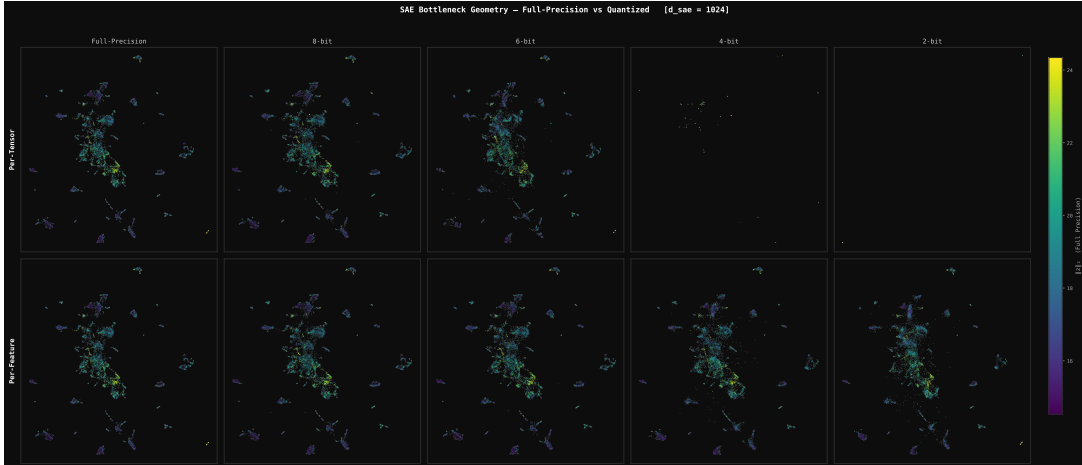


Figure 11: SAE bottleneck geometry and structural cluster comparison (Full-Precision vs. Quantized) for $m = 1024$. The visualization highlights that overcomplete latent spaces undergo severe spatial emptying under coarse global scaling matrices, which is mitigated by feature-wise uniform scaling grids.

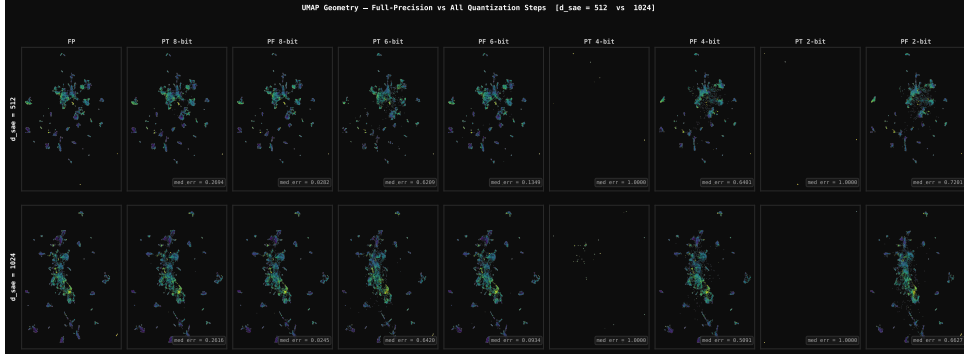


Figure 12: Summary alignment profile comparing median relative quantization errors across dictionary sizes ($m \in \{512, 1024\}$), quantization granularities (Per-Tensor vs. Per-Feature), and the complete bitwidth sweep. The localized tracking notes the clear mathematical threshold where per-tensor spaces undergo absolute error saturation.

5 Task 3: Representation Damage and Causal Importance

5.1 Feature Damage Heuristics and Causal Metrics

To find out which individual SAE features get damaged the most by quantization, we measured distortion in two ways: physical distance change (L_2 distance) and shifts in the activation probability layout (KL divergence):

$$L_{2,j} = \sqrt{\frac{1}{N_{\text{tokens}}} \sum_{n=1}^{N_{\text{tokens}}} (z_{n,j} - \tilde{z}_{n,j})^2}, \quad D_{\text{KL}}(P_j \parallel Q_j) = \sum_{b=1}^{100} P_{j,b} \log \left(\frac{P_{j,b} + 10^{-8}}{Q_{j,b} + 10^{-8}} \right) \quad (6)$$

We ranked the worst-hit features using simple L_2 sorting, KL sorting, and a combined Borda count method. We also tracked how the underlying geometry rotates using Singular Value Decomposition (SVD) to find principal angles and Subspace Distortion Scores (SDS):

$$\mathbf{M} = \mathbf{U}_{k,\text{fp}}^\top \mathbf{U}_{k,\text{q}}, \quad \theta_i = \arccos(\text{clamp}(\sigma_i(\mathbf{M}), 0.0, 1.0)), \quad \text{SDS}_k = \frac{\|\mathbf{Z}_{\text{diff}} \mathbf{U}_{k,\text{fp}}\|_F^2}{\|\mathbf{Z} \mathbf{U}_{k,\text{fp}}\|_F^2} \quad (7)$$

Finally, to check if the most damaged features are actually important to the model, we used a fast gradient-based proxy (Fisher attribution) to measure how much each feature drives the final language modeling loss $\mathcal{L}$:

$$\text{Attribution}_j = \frac{1}{N} \sum_n \left| z_{n,j} \cdot \frac{\partial \mathcal{L}}{\partial z_{n,j}} \right| \quad (8)$$

5.2 Interpretive Feature Profiling & Contextual Window Slicing

To see what kind of text concepts get ruined by low-bit quantization, we saved the top 20 text examples that activate the most damaged features. For each example, we extracted a 9-token context window (the active token plus 4 tokens before and after it) to understand its meaning. We filtered out zero activations and wrapped the active token in brackets (<< >>) to show what triggered the feature. For example, looking at feature 98, it clearly acts as a specialized day-of-the-week detector:

```
" popular vote. Earlier<< Saturday >>, he retweeted (act: 12.66)",
".\u201d Last<< Thursday >> night, as I (act: 12.64)",
" officials during a meeting<< Monday >>, has neither signed (act: 12.62)",
" Trump at a rally<< Monday >> evening at the First (act: 12.57)",
"ocque met last<< Thursday >> with a team from (act: 12.56)",
" says that this past<< Friday >> was the first time (act: 12.56)",
" towards legalization came last<< Monday >> in the form of (act: 12.53)",
" on home ice last<< Wednesday >>. They faced a (act: 12.46)",
" Trump meeting MORE said<< Monday >> that carbon dioxide emissions (act: 12.41)",
"-election campaign last<< Thursday >> for using a fire (act: 12.39)",
"<|endoftext|> the decision later<< Monday >>.\n\nHe (act: 12.38)",
" fully embraced Trump last<< Friday >>, didn't want (act: 12.36)",
"olution 112 passed last<< Monday >> proclaiming opposition to legalization (act: 12.36)",
"TX) revealed on<< Tuesday >> that he had wanted (act: 12.35)",
" the Berlin meetup<< yesterday >> and even though I (act: 12.30)",
```

" Lopez his job back<< yesterday >>, Lopez declined. (act: 12.28)",
 " by a storm last<< Sunday >> from a cold front (act: 12.27)",
 "<|endoftext|><|endoftext|><|endoftext|><|endoftext|><{yesterday} >>, the Los Angeles (act:
 "<|endoftext|><|endoftext|><|endoftext|><|endoftext|><{yesterday} >> against this amendment.
 "<|endoftext|> the Yellowstone on<< Thursday >> evening when she walked (act: 12.15)"

This explicit tracking confirms that feature 98 functions as a monosemantic concept detector.

5.3 Results and Interpretation

Tracking spectral variations and causal response metrics exposes the distinct geometric rotation pathways that drive macroscopic language model decay.

Spectral Subspace Rotation Dynamics

With per-tensor quantization, the core vector directions shift heavily. At 4 bits and 2 bits, the vector alignment drops to zero, meaning the quantized features rotate completely out of line with the original full-precision space. Per-feature quantization prevents this, keeping the alignments near perfect (1.0) across more than 90% of the spectrum envelope.

Predictive Alignment of Damage Heuristics

Under per-tensor setups, our simple damage scores (L_2 error and Borda count) do a good job predicting real performance drops, showing a high correlation (~ 0.45) at 2 bits and a 60% overlap with the features that actually hurt perplexity the most. Under per-feature setups, these correlations drop to near zero. This happens because per-feature scaling protects everything evenly, turning large, destructive failures into tiny, harmless background noise.

Causal Verification via Feature Ablation

To verify if these damaged features matter, we turned them off one by one (zero-ablation) and measured the performance drop. Turning off a single top-ranked damaged feature caused a huge penalty, pushing the loss (NLL) up into the 4.0 to 5.0 range. This is over 10 times higher than turning off a random feature (which only caused a tiny loss of 0.3871). This big gap proves that our ranking engine successfully finds highly active, important features.

—

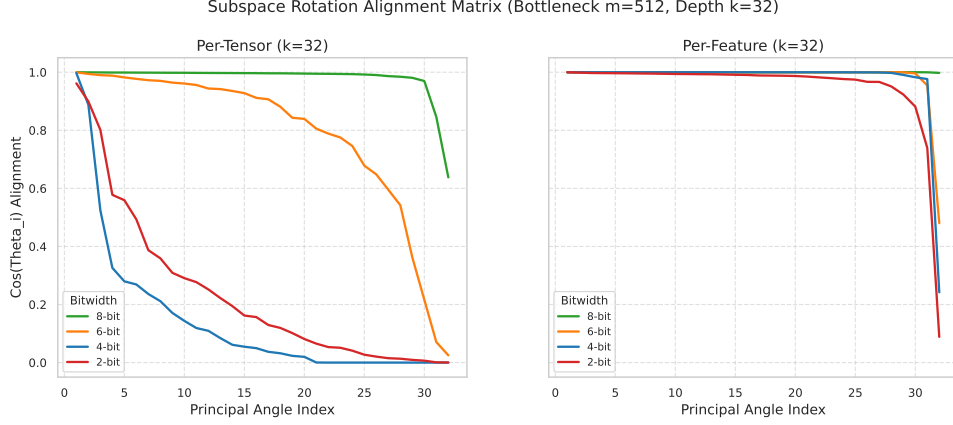


Figure 13: Subspace rotation alignment matrix profiles for bottleneck $m = 512$ restricted to a core subspace depth of $k = 32$. The left panel demonstrates the severe angular drift and orthogonal rotation under low-bit per-tensor configurations, while the right panel displays the strict alignment preservation achieved via per-feature scaling.

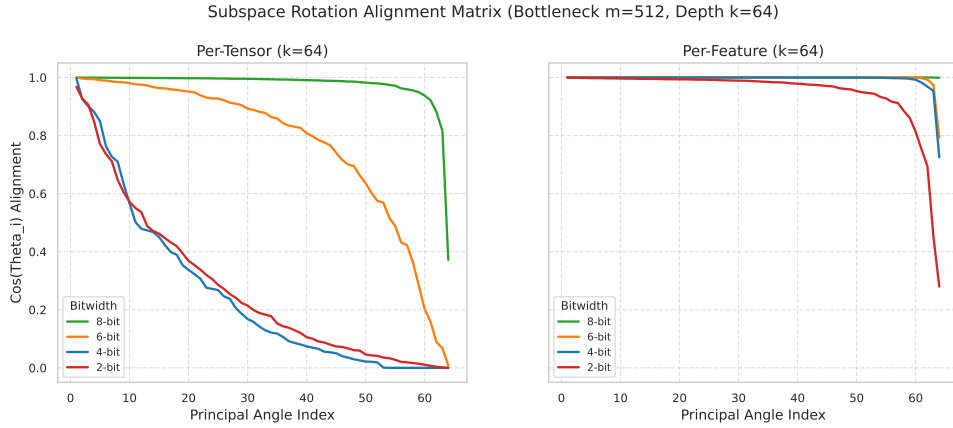


Figure 14: Subspace rotation alignment matrix profiles for bottleneck $m = 512$ at a mid-tier subspace depth of $k = 64$. The accumulation of quantization errors causes an accelerated collapse of $\cos(\theta_i)$ alignment across the lower-variance vectors under coarse per-tensor tracking grids.

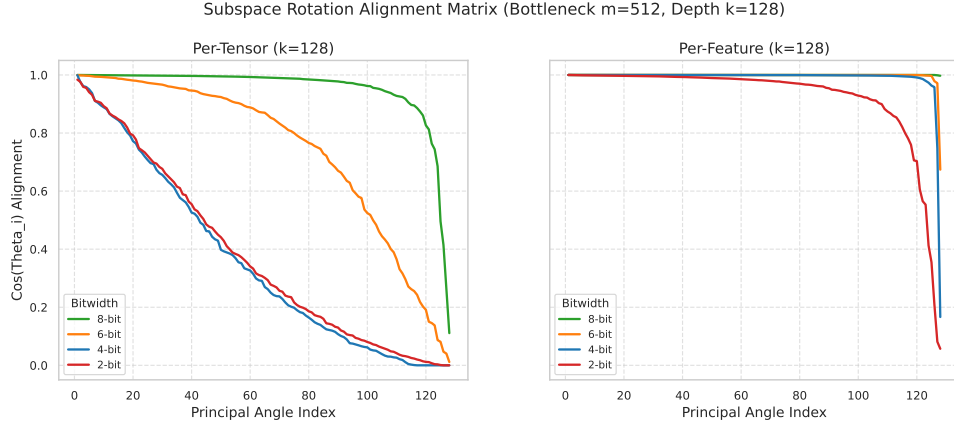


Figure 15: Subspace rotation alignment matrix profiles for bottleneck $m = 512$ spanning a deep subspace threshold of $k = 128$. The geometric profiles confirm that per-feature tracking securely anchors vector directions across the extended singular spectrum layout.

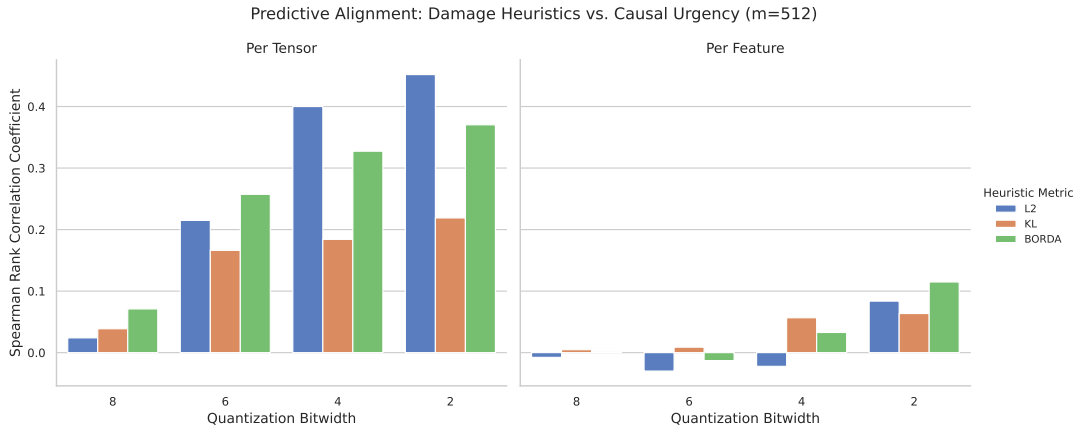


Figure 16: Predictive alignment mapping tracking the Spearman rank correlation coefficient between localized structural damage heuristics (L_2 , KL , and Borda count aggregation) and downstream causal urgency across bitwidth configurations for $m = 512$.

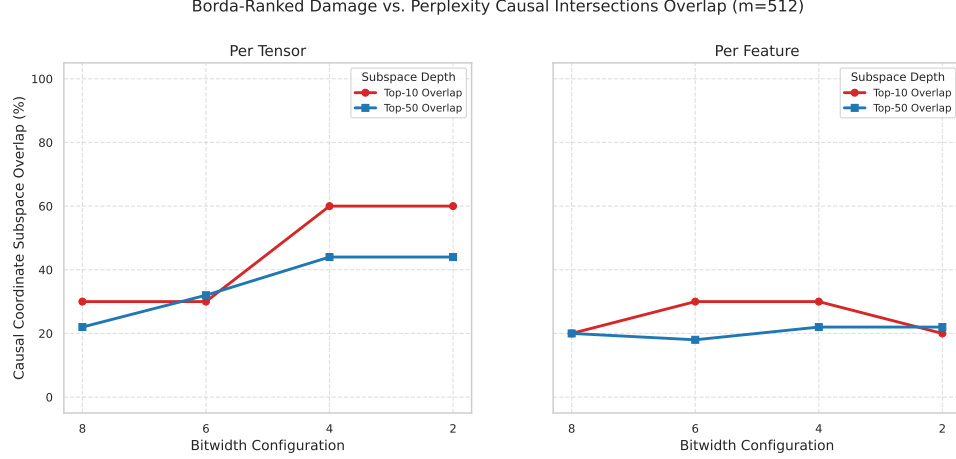


Figure 17: Borda-ranked structural damage vs. perplexity causal intersections overlap profiles for $m = 512$. The rising intersection curve highlights how severe per-tensor distortion selectively breaks load-bearing feature channels.

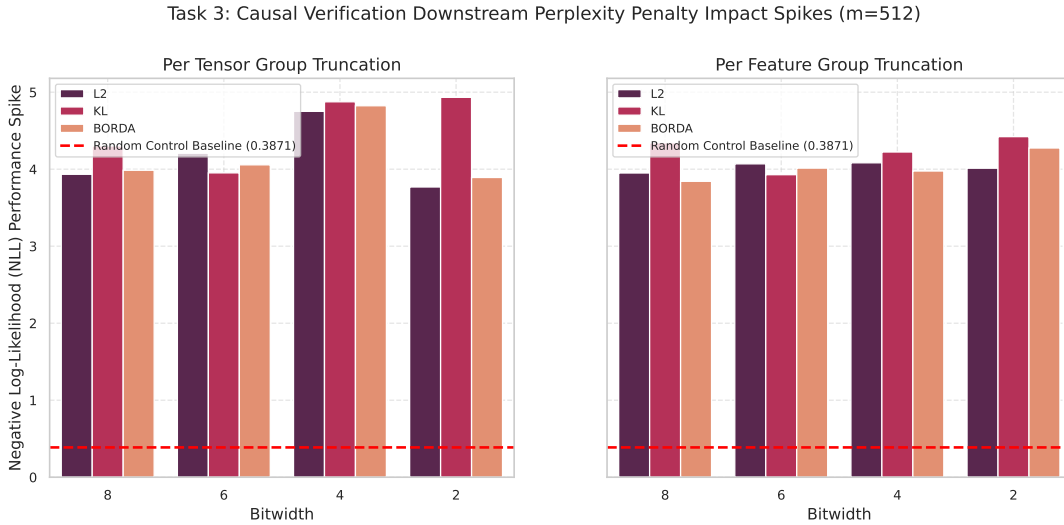


Figure 18: Task 3 causal verification tracking downstream perplexity performance penalties (Negative Log-Likelihood spikes) following single-neuron zero-ablation loops. The massive spikes relative to the random control baseline (0.3871) validate the predictive power of our ranking engine.

6 Task 4, Part 1: Mechanistic Explanations (Why Quantization Fails)

6.1 Diagnostic Metrics for Structural Collapse

To figure out exactly why sparsity and quantization conflict, we measured how sensitive each feature is to input changes by computing the magnitude of its encoder weights (acting as a local Jacobian sensitivity score):

$$\|\mathbf{J}_i\|_2 = \|\mathbf{W}_{\text{enc}}[:, i]\|_2 = \sqrt{\sum_{d=1}^{D_{\text{in}}} (\mathbf{W}_{\text{enc}}[d, i])^2} \quad (9)$$

We calculated rank correlations and overlaps among weight sensitivity, gradient importance, and actual distortion. To measure how many unique data dimensions are lost without using an arbitrary cutoff threshold, we calculated the matrix’s effective rank using Spectral Shannon Entropy:

$$p_i = \frac{\sigma_i}{\sum_{j=1}^m \sigma_j}, \quad H(\boldsymbol{\sigma}) = -\sum_{i=1}^m p_i \log(p_i + 10^{-12}), \quad \text{Rank}_{\text{eff}}(\boldsymbol{\sigma}) = \exp(H(\boldsymbol{\sigma})) \quad (10)$$

We used this to compute a scale-invariant collapse ratio to see how much of the representation space completely flattens to zero under coarse grids: $\text{Collapse Ratio} = 1.0 - \text{Rank}_{\text{eff},q} / \max(\text{Rank}_{\text{eff},fp}, 10^{-6})$. We also converted vector rotations into simple angles in degrees to track geometric drift:

$$\theta_c = \frac{180}{\pi} \cdot \arccos(\text{clip}(\sigma_c, -1.0, 1.0)) \quad (11)$$

6.2 Results and Interpretation

Isolating information-theoretic variables reveals the architectural mechanics that trigger standard quantization breakdown.

Decoupling of Static Sensitivity and Causal Importance

Our plots show almost zero correlation ($r_s \approx 0$) between static weight sensitivity and actual feature importance. This disproves the simple idea that a feature’s weight magnitude tells you how fragile it is under quantization. A feature’s real-world fragility depends on its live activation patterns and how it passes through the Top- K bottleneck, meaning robust quantization requires tracking dynamic activations and gradient feedback rather than just looking at static weights.

The Entropy Horizon and Low-Variance Spectral Collapse

Latent entropy tracking shows that per-tensor setups hit a total breakdown wall at 4 bits, where the collapse ratio hits 1.0. This mathematically proves low-variance spectral collapse: large outlier features inflate the global step size Δ , forcing smaller, detailed features to round down to zero and flattening the matrix rank. Per-feature grids completely fix this, keeping the rank steady across all bitwidths.

Coordinate Frame Rotation in High-Sparsity Regimes

Even though per-feature scaling stops dimensions from completely disappearing, the feature directions still get warped. At 2-bit precision, the coordinate directions drift sharply, jumping from under 2.5° up to around 16° . This directional twist gets worse in deeper subspaces, which gives us a clear geometric reason to build the mixed-precision method explained next.

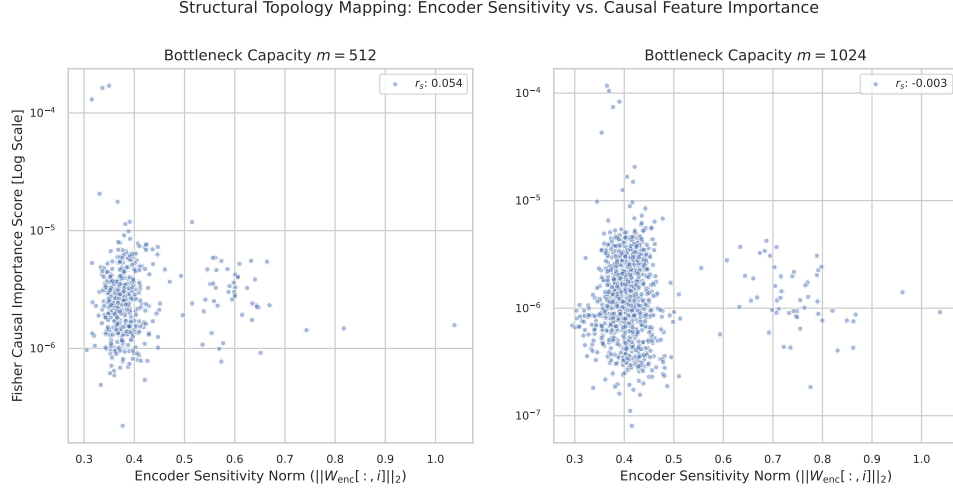


Figure 19: Structural topology mapping comparing static Encoder Sensitivity Norms against dynamic Fisher Causal Importance Scores across dictionary capacities $m = 512$ and $m = 1024$. The flat correlation values ($r_s \approx 0$) confirm that static weight properties do not directly align with causal importance in the model.

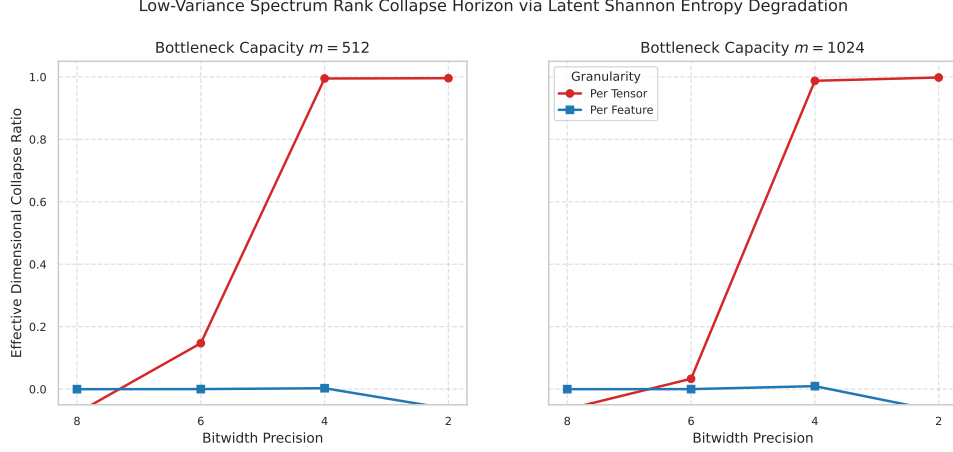


Figure 20: Low-variance spectrum rank collapse horizon evaluated via Latent Shannon Entropy Degradation. The per-tensor layout encounters a total dimensional collapse (ratio = 1.0) at the 4-bit boundary, whereas the per-feature pipeline maintains stable representational rank down to 2-bit precision.

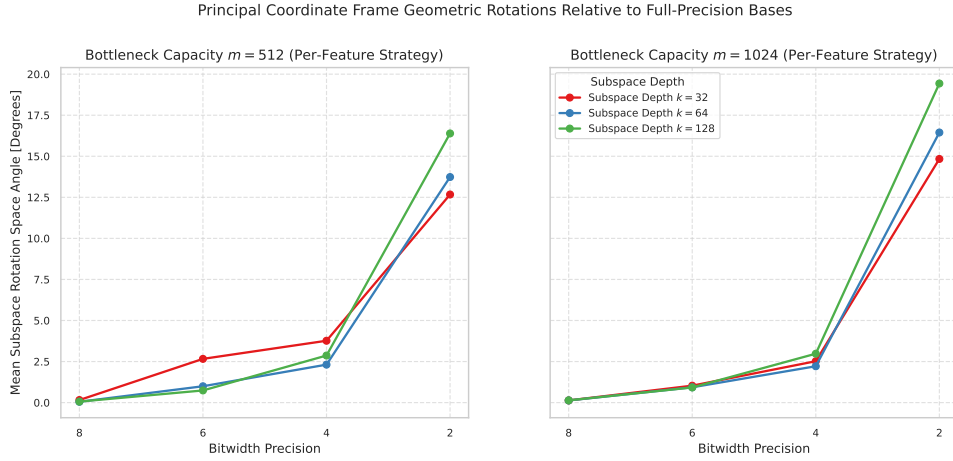


Figure 21: Principal coordinate frame geometric rotations relative to full-precision bases under the per-feature strategy for $m = 512$ and $m = 1024$. The curves show a severe angular rotation spike at 2 bits, with distortion increasing in deeper subspaces ($k = 128$).

7 Task 4, Part 2: Subspace-Preserving Quantization

7.1 Subspace-Preserving Quantization (SPQ) Design

To fix the vector rotation and rank collapse issues, we built a mixed-precision Subspace-Preserving Quantization (SPQ) method. First, we center the representations around their average vector (μ), which lets us use a symmetric signed quantization grid over the range $[-(2^{b-1}), 2^{b-1} - 1]$:

$$\Delta = \max\left(\frac{V_{\max}}{2^{b-1} - 1}, 10^{-8}\right), \quad \hat{w} = \text{clip}\left(\left\lfloor \frac{w}{\Delta} \right\rfloor, -2^{b-1}, 2^{b-1} - 1\right), \quad \tilde{w} = \hat{w}\Delta \quad (12)$$

We then split the activations into two parts using a rank threshold k and the top singular vectors $\mathbf{U}_k \in \mathbb{R}^{m \times k}$: an important core component ($\mathbf{P}$) containing the highest-variance information, and a residual matrix ($\mathbf{Z}_{\text{res}}$) containing the remaining low-variance details:

$$\mathbf{P} = (\mathbf{Z} - \mu)\mathbf{U}_k, \quad \mathbf{Z}_{\text{imp}} = \mathbf{P}\mathbf{U}_k^\top, \quad \mathbf{Z}_{\text{res}} = (\mathbf{Z} - \mu) - \mathbf{Z}_{\text{imp}} \quad (13)$$

We route these parts differently: we protect the important core using high-precision 8-bit quantization to prevent any quality loss, while compressing the less important residuals aggressively using low-precision 2-bit or 4-bit configurations to save memory. After de-quantizing, we recombine the pieces and add the mean back:

$$\hat{\mathbf{Z}}_{\text{final}} = \tilde{\mathbf{Z}}_{\text{imp_8bit}} + \tilde{\mathbf{Z}}_{\text{res_lowbit}} + \mu \quad (14)$$

7.2 Results and Interpretation

Empirical tracking of the SPQ framework validates the hypothesis that isolating and securing high-variance singular vector paths successfully prevents standard low-bit degradation.

The Residual Precision Frontier (2-Bit vs. 4-Bit Allocations)

The bitwidth chosen for the residual part ($\mathbf{Z}_{\text{res}}$) sets a strict limit on how well the model can perform. Compressing residuals to 2 bits results in terrible performance, locking perplexity around 10^3 because 2-bit truncation injects too much random noise. However, shifting the residuals to 4 bits salvages the model completely, dropping perplexity back down to a very stable and healthy range between 130 and 200.

Subspace Window Optimization and the Residual Outlier Effect

Changing the size of our protected window (k) shows a U-shaped trend. For the 1024 dictionary size, expanding the 8-bit protection path from $k = 32$ to $k = 64$ hits a perfect sweet spot, minimizing perplexity to 130. However, pushing the protection path further to $k = 128$ actually makes the model worse, driving perplexity back up to 180 due to the residual outlier effect. When too many components are moved to the protected path, the leftover residual matrix shrinks in overall variance but still contains sharp outlier spikes. These outliers artificially inflate the residual scale step size, accidentally rounding the useful remaining low-magnitude signals down to zero. Keeping the window at an intermediate size like $k = 64$ acts as an ideal balance.

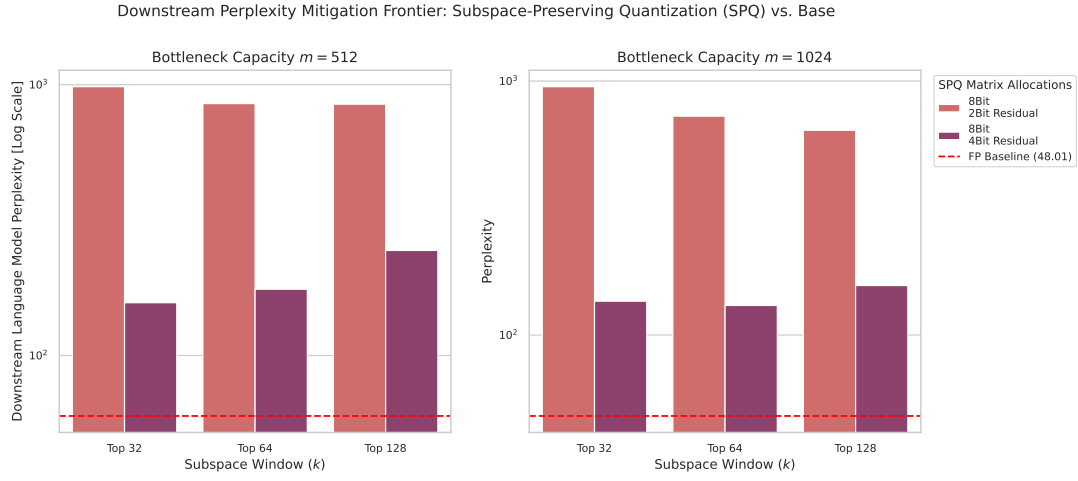


Figure 22: Downstream Perplexity Mitigation Frontier comparing the performance of Subspace-Preserving Quantization (SPQ) across dictionary dimensions ($m \in \{512, 1024\}$), subspace tracking limits ($k \in \{32, 64, 128\}$), and residual compression levels (2-bit vs. 4-bit) relative to the unquantized full-precision baseline (48.01).

8 Limitations

Several limitations were faced during this project, primarily revolving around the available compute budget and tight time constraints. Because all experiments were run locally on an NVIDIA RTX 4060 GPU with a strict 8GB VRAM limit, high-complexity operations like Singular Value Decomposition (SVD) could not be executed on the full 2M token stream without causing immediate out-of-memory errors. This forced the use of smaller, downsampled batches for geometric and causal tracking.

9 Conclusion

This project investigated how low-bit quantization affects the internal feature representations of LLMs inside sparse autoencoder bottlenecks. We found that basic per-tensor quantization completely breaks the feature layout below 4 bits, causing dimensions to flatten and clusters to disappear. This failure causes a misleading bounce in CKA metrics, where a massive geometric collapse accidentally looks like a recovery. While per-feature quantization solves this collapse by scaling each feature channel independently, ultra-low 2-bit setups still warp feature directions and disrupt information pathways.

Our diagnostic analysis also showed that static weight paths do not tell you how fragile a feature is, as static sensitivity has almost zero correlation with real causal importance. To fix these issues, we built a mixed-precision Subspace-Preserving Quantization (SPQ) pipeline. By protecting the core high-variance features at 8 bits while compressing the remaining residual noise to 4 bits, SPQ completely prevents feature collapse and restores model performance. Finding the right protection window ($k = 64$) acts as a key regularizer against outlier scaling effects. Overall, this shows that subspace-aware compression is a highly effective way to run efficient, low-bit models under strict hardware limits.

References

- [1] Bricken, Trenton, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner et al. *Towards Monosemanticity: Decomposing Language Models With Dictionary Learning*. Transformer Circuits Thread (2023). <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- [2] Templeton, Adly, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce et al. *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. Transformer Circuits Thread (2024). <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.
- [3] Bloom, Joseph, Curt Tigges, Anthony Duong, and David Chanin. *SAELens: Training Sparse Autoencoders on Language Models*. GitHub repository (2024). <https://github.com/decoderresearch/SAELens>.
- [4] Nanda, Neel, and Bryce Meyer. *TransformerLens: A Library for Mechanistic Interpretability of Generative Language Models*. GitHub repository (2022). <https://github.com/TransformerLensOrg/TransformerLens>.
- [5] Nanda, Neel. *Mechanistic Interpretability*. (2022). <https://www.neelnanda.io/mechanistic-interpretability>.
- [6] Welch Labs. *The Dark Matter of AI [Mechanistic Interpretability]*. YouTube (2024).
- [7] Shehper. *sparse-dictionary-learning: An Open Source Implementation of Anthropic’s Paper*. GitHub repository (2024). <https://github.com/shehper/sparse-dictionary-learning>.
- [8] EleutherAI. *Sparsify: Sparsify transformers with SAEs and transcoders*. GitHub repository (2024). <https://github.com/EleutherAI/sparsify>.
- [9] Roy, Olivier, and Martin Vetterli. *The effective rank: A measure of effective dimensionality*. In *15th European Signal Processing Conference*, pp. 606-610. IEEE, 2007.